

DoseMate iOS and tvOS Application Development Report

SE4041 Mobile Application Design and Development Assignment 2 Swift

Student Name	Dushmantha Rathnayake
Student ID	It23285460
Programme	BSc Honours in Information Technology
Module	SE4041 Mobile Application Design and Development
Academic Year	2026
Submission	Part A Advanced iOS App and Part B tvOS Prototype

Repository: github.com/du5hd1v/MedicineReminder

Classification: Public-SLIIT

Declaration

I declare that this report describes the DoseMate applications submitted for SE4041 Assignment 2. The implementation was completed as an individual project. Any external frameworks, development tools, and AI-assisted support used during development are identified in this report. The final design decisions, code review, testing, and responsibility for the submitted work remain with the student.

Abstract

This report presents the design and development of DoseMate, a medicine reminder ecosystem consisting of a functional SwiftUI iOS application and a tvOS family dashboard prototype. The iOS application addresses the real-world problem of missed or incorrectly tracked medicine doses. It allows a user to create, edit, pause, delete, and complete medicine reminders; records dose history; monitors remaining supply; persists data locally; and schedules recurring local notifications. The tvOS prototype extends the concept into a shared household environment by displaying family profiles, daily completion progress, upcoming doses, and urgent reminders on Apple TV. Both applications use Swift and SwiftUI, with reusable models, focused navigation, responsive layouts, platform-appropriate controls, and consistent DoseMate branding.

The report explains the problem definition, audience, requirements, architecture, data model, user interface, implementation decisions, persistence, notification integration, testing approach, performance considerations, privacy, limitations, business potential, and future development. It also maps the delivered work to the Part A and Part B assessment criteria and documents how AI assistance supported development.

Keywords

SwiftUI, iOS, tvOS, medicine reminder, local notifications, data persistence, accessibility, family health dashboard

Classification: Public-SLIIT

Table of Contents

1 Introduction

2 Problem Definition and Project Objectives

3 Requirements Analysis

4 Part A iOS Application Design

5 Part A Technical Implementation

6 User Interface and User Experience

7 Data Persistence and Notifications

8 Testing Debugging and Performance

9 Part B tvOS Prototype

10 Innovation Market Potential and Business Strategy

11 Project Management Version Control and AI Use

12 Challenges Limitations and Future Work

13 Requirement Traceability

14 Conclusion

References

Appendices

Classification: Public-SLIIT

1 Introduction

Mobile health applications can reduce the everyday friction involved in following a medicine routine. A user may need to remember several medicines at different times, understand dosage instructions, monitor remaining supply, and communicate progress to family members. Missed doses may occur because reminders are not visible, schedules are stored in different places, or the user cannot quickly confirm whether a dose was taken.

DoseMate was designed as a focused response to this problem. Part A is an iOS application for personal medicine management. Part B is a tvOS prototype that explores how the same product concept can support shared household awareness through a television interface. The two applications form a coherent product family while demonstrating different interaction models and screen contexts.

1.1 Project Scope

- Develop a SwiftUI iOS application with more than three navigable screens.
- Support medicine creation, editing, deletion, activation, completion tracking, supply tracking, and history.
- Use local notifications to deliver recurring medicine reminders.
- Save medicine records locally and restore them when the application restarts.
- Create a branded, accessible, responsive interface with reusable SwiftUI components and animation.
- Develop a separate tvOS prototype designed for Apple TV focus navigation and distance viewing.
- Document the design, implementation, testing, market potential, AI assistance, and future work.

1.2 Deliverables

Table 1 Project deliverables

Deliverable	Platform	Purpose	Status
DoseMate	iOS	Personal medicine reminders and adherence tracking	Implemented
DoseMate TV	tvOS	Shared household medicine dashboard prototype	Implemented prototype
GitHub repository	GitHub	Version-controlled source and documentation	Published
Project report	Word document	Design, implementation, testing, and reflection	Completed

Classification: Public-SLIIT

2 Problem Definition and Project Objectives

2.1 Real World Problem

Medicine routines are repetitive but important. Users may forget a scheduled dose, confuse one medicine with another, fail to remember whether a dose was already taken, or discover too late that their supply is running low. Generic alarms provide a time signal but do not store dosage, medicine type, notes, stock count, or adherence history. DoseMate combines these related needs in a single application.

2.2 Purpose

The purpose of DoseMate is to help users maintain a clear and consistent medicine routine. The application presents the day's active medicines in time order, sends a recurring notification, allows the user to record completion, updates the remaining supply, and preserves a history that can be reviewed later. DoseMate TV adds shared visibility for households where more than one person participates in care.

2.3 Target Audience

Table 2 Target audiences and design responses

Audience	Need	Design Response
Adults with regular medication	Reliable daily reminders and simple completion tracking	Chronological Today screen and one-tap Mark Taken action
Older users	Readable controls and reduced interaction complexity	Large labels, familiar symbols, clear contrast, and simple navigation
Students and busy workers	Fast entry and quick status checking	Compact form, summary cards, and notifications
Caregivers and families	Shared awareness of medicine schedules	tvOS family overview and member schedule screens
Users managing multiple medicines	Identification, history, and stock awareness	Categories, dosage, notes, weekly counts, and supply warnings

2.4 Objectives

1. Provide a clear daily overview of active medicine reminders.
2. Allow complete medicine record management through create, read, update, and delete operations.
3. Send recurring local notifications at the selected reminder time.
4. Record completion history and calculate daily and weekly progress.
5. Track remaining doses and visually indicate low supply.
6. Apply consistent branding, animation, accessibility labels, and platform-native navigation.

Classification: Public-SLIIT

7. Demonstrate a tvOS-specific extension with large focusable controls and household-level visibility.
8. Maintain the source through Git version control and provide reproducible build instructions.

3 Requirements Analysis

3.1 Functional Requirements

Table 3 Functional requirements

ID	Requirement	Implementation
FR01	View today's active medicine schedule	TodayView displays active records ordered by hour and minute
FR02	Add a medicine	AddMedicineView captures name, dosage, category, supply, time, and notes
FR03	Edit an existing medicine	EditMedicineView reuses MedicineFormView and updates the selected model
FR04	Delete a medicine	Swipe action and detail confirmation remove the record and notification
FR05	Mark or undo a taken dose	MedicineStore adds or removes today's date and adjusts supply
FR06	Pause or resume reminders	The isActive property and notification manager control reminder state
FR07	Review adherence history	HistoryView lists total, weekly, and recent taken dates
FR08	Persist data	Medicine records are encoded as JSON and saved through UserDefaults
FR09	Receive reminders	UNUserNotificationCenter schedules repeating calendar notifications
FR10	View family medicine status on TV	DoseMate TV presents member progress and upcoming doses
FR11	Navigate with Apple TV focus	Focusable NavigationLink and card controls support the remote

3.2 Non Functional Requirements

Table 4 Non functional requirements

Category	Requirement	Response
Usability	Common tasks should require few steps	Tab navigation, reusable forms, and one-tap actions
Responsiveness	UI must update immediately after state changes	ObservableObject and published medicine array
Reliability	Data should survive application restart	Codable JSON persistence

Classification: Public-SLIIT

Accessibility	Controls should remain understandable with assistive technology	Semantic labels, SF Symbols, Dynamic Type compatible text
Performance	Lists and calculations should remain lightweight	Small local data model and computed summary properties
Privacy	Medicine information should remain on the device	No network transmission or external analytics
Maintainability	Views and responsibilities should be separated	Model, store, notification service, and screen files
Portability	Prototype must use platform-native frameworks	SwiftUI for both iOS and tvOS

4 Part A iOS Application Design

4.1 Application Structure

The iOS application follows a lightweight Model View Store Service structure. Medicine.swift defines the domain model and category metadata. MedicineStore.swift owns application state and business rules. SwiftUI view files render state and forward user actions to the store. NotificationManager.swift isolates the UserNotifications API. This separation keeps platform services out of the presentation code and makes the main user flows easy to follow.

Table 5 iOS application architecture

Layer	Main Files	Responsibility
Application	MedicineReminderApp.swift, RootView.swift	Create shared store, request permission, show splash, and provide tab navigation
Presentation	TodayView, AddMedicineView, MedicineDetailView, HistoryView	Display content, collect input, and navigate between screens
Reusable UI	MedicineCardView, StatTile, MedicineFormView	Provide consistent cards, statistics, and form controls
State and logic	MedicineStore.swift	CRUD operations, sorting, progress, completion, supply, and persistence
Domain model	Medicine.swift	Represent medicine records, categories, formatted time, and derived history values
Platform service	NotificationManager.swift	Request authorization and schedule or cancel notifications
Resources	Assets.xcassets	App icon, logo, accent color, dark and tinted icon variants

Classification: Public-SLIIT

4.2 Navigation Design

RootView initially presents the branded splash screen and then transitions to MainTabView. The tab bar contains Today, Add, and History. Each tab uses NavigationStack so deeper screens can be pushed without losing the user's place. Selecting a medicine card opens MedicineDetailView. Selecting Edit presents a modal editing form, while the Back action dismisses the sheet. This combines persistent top-level navigation with focused task flows.

4.3 Screen Design

Table 6 iOS screens

Screen	Purpose	Key Components
Splash	Introduce branding during initial loading	Logo, app name, gradient, scale animation, ProgressView
Today	Show current adherence status and schedule	Hero summary, ProgressView, statistics, medicine cards, reset control
Add Medicine	Create a new reminder	Form, TextField, Picker, Stepper, DatePicker, validation, confirmation alert
Medicine Details	Inspect and manage one medicine	Sections, status labels, mark taken, pause, edit, delete confirmation
Edit Medicine	Update an existing record	Reusable MedicineFormView with prefilled state
History	Review adherence information	Summary tiles, weekly count, recent completion dates, detail links

5 Part A Technical Implementation

5.1 Medicine Model

Medicine is an Identifiable, Codable, and Equatable structure. A UUID uniquely identifies each item and is also used as the notification request identifier. Stored properties include name, dosage, notes, reminder hour and minute, category, remaining supply, active status, creation date, and an array of completion dates. Codable enables the complete array to be encoded and decoded without manual field mapping.

Table 7 Medicine data model

Property	Type	Purpose
id	UUID	Stable identity for SwiftUI, updates, deletion, and notification cancellation
name	String	User-facing medicine name

Classification: Public-SLIIT

dosage	String	Dose instruction such as 5 mg or 1 tablet
notes	String	Additional instructions such as take after food
hour and minute	Int	Daily reminder time components
category	MedicineCategory	Type-specific icon and color
supplyCount	Int	Remaining number of doses
isActive	Bool	Whether the reminder is currently enabled
takenDates	Date array	Completion history used for daily and weekly calculations

5.2 State Management

MedicineStore is created once in MedicineReminderApp with StateObject and injected through environmentObject. Views observe the same store, so changes made on the add, edit, detail, or Today screens appear immediately across the application. The medicines array is published and automatically saved in didSet. The store is isolated to the main actor because UI state must be updated on the main thread.

5.3 Business Rules

- Active medicines are sorted chronologically by hour and minute.
- Daily progress equals completed active medicines divided by the active medicine count.
- Marking a dose as taken stores the current date and decreases supply when supply is above zero.
- Undoing a taken dose removes today's completion and restores one unit of supply.
- Editing a medicine cancels the old notification and schedules a replacement when the reminder remains active.
- Deleting a medicine cancels its pending notification before removing the record.
- Supply values are constrained to zero or greater.
- The save button remains disabled until both name and dosage contain non-whitespace text.

5.4 Reusable Components

MedicineFormView is reused for add and edit workflows, reducing duplication and preserving consistent validation and field order. MedicineCardView renders a compact schedule record and exposes an action closure for completion. StatTile is shared by Today and History to present numeric summaries. MedicineCategory centralizes display labels, SF Symbols, and colors for each medicine type.

Classification: Public-SLIIT

6 User Interface and User Experience

6.1 Visual Identity

DoseMate uses teal as its primary accent because it communicates clarity and healthcare without relying on an overly clinical appearance. Green indicates completed doses, red indicates urgent or low-supply states, and each medicine category has a distinct supporting color. The generated application icon and DoseMate logo are included in the asset catalog with standard, dark, and tinted variants.

6.2 Advanced SwiftUI Elements

Table 8 Advanced interface elements

Element	Where Used	Benefit
Animated transition	Splash to main interface	Creates a smooth branded launch experience
Repeating scale animation	Splash logo	Provides visible loading feedback without a complex panel
Spring animation	Mark Taken action	Makes completion feel immediate and responsive
Swipe action	Today medicine cards	Provides a familiar fast deletion gesture
Progress views	Today and tvOS member cards	Communicate adherence at a glance
Adaptive stacks and grids	iOS summaries and tvOS dashboard	Organize information for the target screen size
Confirmation dialog	Delete Medicine	Prevents accidental destructive action
ContentUnavailableView	Empty or removed states	Provides clear feedback instead of a blank screen

6.3 Accessibility

The application relies on system fonts so text supports Dynamic Type. System colors and semantic backgrounds adapt to Light and Dark Mode. Important icon-only buttons include accessibility labels, including Mark Taken, Undo Taken, and Reset Demo Data. SF Symbols reinforce meaning without replacing text in critical workflows. The tvOS interface uses large text, generous spacing, strong contrast, and focusable controls for distance viewing.

6.4 User Flow

1. The user opens DoseMate and sees a short branded loading screen.
2. The Today screen presents active medicines, daily progress, and total completion statistics.
3. The user selects Add, enters medicine information, chooses a category and time, and saves the record.
4. DoseMate stores the record and schedules a daily notification.

Classification: Public-SLIIT

5. At the dose time, the device presents the notification.
6. The user opens DoseMate and marks the medicine as taken.
7. The application adds a history entry, updates progress, and reduces remaining supply.
8. The user can review History or open Details to edit, pause, or delete the medicine.

7 Data Persistence and Notifications

7.1 Local Persistence

The current application persists medicine data by encoding the medicine array with JSONEncoder and saving the result under a versioned UserDefaults key. At launch, MedicineStore attempts to load the current key and then a legacy key for backward compatibility. If data is unavailable or decoding fails, sample records are loaded so the application remains demonstrable.

This approach is suitable for a compact prototype because the dataset is small, local, and structurally simple. For production use, SwiftData or Core Data would provide stronger query, migration, relationship, and scalability support. The report therefore treats the current mechanism as working local persistence and identifies migration to SwiftData as priority future work.

7.2 Local Notification Integration

NotificationManager uses UserNotifications. Permission for alerts, badges, and sound is requested during the root view task. A repeating UNCalendarNotificationTrigger is built from the selected hour and minute. The notification title contains the medicine name, while the body includes dosage and notes. Each request uses the medicine UUID, allowing the application to cancel or replace the exact reminder when the user edits, pauses, or deletes a record.

7.3 Privacy and Security

- Medicine information is stored locally and is not transmitted to a server.
- The application requests only notification permission required by the implemented feature.
- No advertising identifier, analytics SDK, account creation, or location tracking is used.
- Notification text may be visible on the lock screen according to the user's system settings.
- A production healthcare release should add a privacy notice, optional device authentication, encrypted synchronization, and clinical disclaimers.

8 Testing Debugging and Performance

8.1 Testing Strategy

Testing combined build verification, simulator/device interaction, functional scenario checks, visual inspection, and edge-case review. The iOS project was built and exercised in Xcode, while the tvOS project was compiled against the AppleTVOS SDK. The following functional cases define the expected acceptance behaviour.

Classification: Public-SLIIT

Table 9 Functional test cases

Test	Action	Expected Result	Result
T01 Launch	Open the iOS app	Splash appears, then Today screen loads	Pass
T02 Add	Save valid medicine details	Record appears and confirmation alert is shown	Pass
T03 Validation	Leave name or dosage empty	Save button remains disabled	Pass
T04 Completion	Tap Mark Taken	Status, progress, history, and supply update	Pass
T05 Undo	Undo today's completion	History entry is removed and supply restored	Pass
T06 Edit	Change time and dosage	Record updates and reminder is replaced	Pass
T07 Pause	Turn reminder off	Record is inactive and pending notification is cancelled	Pass
T08 Delete	Confirm delete	Record and notification are removed	Pass
T09 Persistence	Restart after making changes	Saved records are restored	Pass
T10 Low supply	Set supply to 3 or lower	Supply warning uses red emphasis	Pass
T11 Empty state	Disable or remove all medicines	No active medicines message is shown	Pass
T12 tvOS build	Compile DoseMate TV for tvOS	Project compiles successfully	Pass
T13 tvOS navigation	Select a family card	Member schedule opens using focus navigation	Pass by code review/build

8.2 Debugging

Development issues included simulator runtime installation, signing configuration, keychain access for code signing, and project build destinations. These were separated from source-code errors by testing generic platform builds and checking Xcode's Signing and Capabilities settings. The tvOS target was verified with code signing disabled for compilation, confirming that its Swift source and resources build successfully.

8.3 Performance Considerations

- The application stores a small value-type model and does not perform network requests.
- Computed progress values iterate over the medicine array, which is appropriate for the expected small dataset.
- SwiftUI updates only views observing the shared published store.

Classification: Public-SLIIT

- Notification scheduling is performed only when records are added, edited, enabled, disabled, deleted, or reset.
- Images are stored in the asset catalog and rendered at fixed presentation sizes.
- For larger datasets, history should be queried from SwiftData rather than repeatedly filtering an in-memory date array.

8.4 Recommended Automated Tests

The current repository does not include dedicated XCTest targets. To strengthen production readiness, the next iteration should add unit tests for progress calculations, weekly counts, supply adjustment, persistence decoding, and reminder state. UI tests should cover tab navigation, form validation, add/edit/delete flows, empty states, and accessibility identifiers. This is an acknowledged limitation of the present submission.

9 Part B tvOS Prototype

9.1 Concept

DoseMate TV is a shared family medicine dashboard designed specifically for Apple TV. While the iOS application supports an individual's detailed interaction, the television prototype provides a high-visibility household overview. A family can see daily progress, upcoming doses, urgent items, and separate schedules for multiple members.

9.2 Platform Rationale

Apple TV is a shared device located in a common space. This makes it suitable for passive awareness and coordinated care. The design uses a landscape-first layout, large typography, high contrast, generous spacing, focusable cards, and simple directional navigation. These choices respond to the viewing distance and input model of tvOS.

9.3 tvOS Architecture

Table 10 tvOS project structure

File	Responsibility
DoseMateTVApp.swift	Application entry point and root window
TVModels.swift	FamilyMember, TVMedicine, TVDosePlan, computed progress, and sample prototype data
TVHomeView.swift	Dashboard, family cards, upcoming doses, member schedules, and platform feature cards
TVPitchView.swift	Page-based presentation of problem, solution, target market, revenue, and launch strategy
Assets.xcassets	Accent color resources

Classification: Public-SLIIT

README.md and PITCH.md	Build guidance, concept rationale, comparison, business model, and demo structure

9.4 Core Prototype Features

- Family Today cards show each member, role, completion percentage, and progress bar.
- NavigationLink cards open a dedicated member schedule using Apple TV focus interaction.
- Upcoming dose cards display time, medicine, dosage, completion state, and urgent Soon status.
- The interface uses a dark high-contrast gradient suitable for television viewing.
- A built-in Pitch screen uses page navigation to communicate the business concept during presentation.
- Sample data demonstrates parents, a student, different medicine types, completed doses, and urgent reminders.

9.5 Similar Applications and Differentiation

Table 11 Concept comparison

Product Type	Typical Strength	Gap Addressed by DoseMate TV
Mobile medicine reminder apps	Detailed personal reminders and logs	Usually private to one phone and not visible to the household
Apple Health	Broad personal health data aggregation	Not designed as a shared television medicine dashboard
General reminder applications	Flexible task and alarm creation	Do not provide medicine categories, supply, adherence, or family view
DoseMate TV	Shared large-screen medicine status	Combines household visibility, focus navigation, urgent doses, and member progress

10 Innovation Market Potential and Business Strategy

10.1 Value Proposition

DoseMate provides a connected product concept across personal and shared screens. The iPhone remains the private, portable interaction point, while Apple TV becomes a household awareness surface. The key value proposition is simple coordination: the right medicine information is visible in the context where an individual or family needs it.

10.2 Market Segments

- Families supporting older relatives who have several daily medicines.
- Caregivers who need a quick visual summary of household routines.
- Shared homes where responsibility is distributed between family members.

Classification: Public-SLIIT

- Students and professionals who want a structured personal reminder application.
- Community care organizations and pharmacies interested in adherence support.

10.3 Revenue Model

Table 12 Proposed revenue model

Tier or Channel	Offering	Potential Revenue
Free	Personal reminders, basic history, and limited family profiles	User acquisition
Premium subscription	Cloud sync, unlimited profiles, caregiver alerts, reports, and refill forecasting	Recurring monthly or annual fee
Family plan	Shared iOS and tvOS access for one household	Higher-value subscription
Pharmacy partnerships	Opt-in refill links and educational campaigns	Referral or campaign fee
Institutional licensing	Configured dashboards for care facilities	Business licensing

10.4 Go To Market Strategy

1. Launch the iOS application as the primary personal product and validate reminder and history workflows.
2. Introduce the tvOS dashboard as a companion experience for family plans.
3. Recruit test households and caregivers to evaluate readability, trust, and coordination benefits.
4. Publish educational content focused on routine support while avoiding medical treatment claims.
5. Develop opt-in relationships with pharmacies and community health programs.
6. Add secure synchronization and caregiver notifications after privacy and security review.

11 Project Management Version Control and AI Use

11.1 Development Approach

The project was developed incrementally. The initial SwiftUI application was created in Xcode, then the medicine model, shared store, screens, notifications, branding, splash experience, and tvOS prototype were added in stages. Build checks were performed after significant changes. The repository contains separate iOS and tvOS Xcode projects so each platform can be opened and demonstrated independently.

11.2 Version Control

Git is used for source control and the main branch is pushed to the GitHub repository at github.com/du5hd1v/MedicineReminder. The repository includes a .gitignore that excludes macOS metadata, DerivedData, build output, and Xcode user-interface state. The visible history includes the

Classification: Public-SLIIT

initial project commit and a separate commit that adds the Part B tvOS prototype. A stronger production workflow would use feature branches and pull requests for each major capability.

11.3 AI Assisted Development Disclosure

AI assistance was used as a development support tool. It helped interpret the assignment brief, generate initial SwiftUI structures, diagnose Xcode setup and signing issues, create branding assets, improve the splash screen, organize the tvOS prototype, prepare Git commands, and structure this report. The student reviewed the generated work, ran the projects in Xcode, selected the final design, and remains responsible for understanding and defending the submission.

Table 13 AI assistance record

AI Assisted Area	Example Assistance	Student Responsibility
Planning	Convert rubric into app features and deliverables	Confirm scope and select the DoseMate concept
Implementation	Suggest SwiftUI views, model separation, and notification flow	Review code, build, test, and explain decisions
Design	Generate matching logo and propose interface styling	Choose final branding and verify usability
Troubleshooting	Interpret build, simulator, Git, and signing messages	Perform account access and device authorization
Documentation	Organize technical report and rubric mapping	Verify accuracy and add personal reflection/student details

12 Challenges Limitations and Future Work

12.1 Development Challenges

Table 14 Development challenges

Challenge	Impact	Resolution
New Xcode environment	Project creation and simulator setup were unfamiliar	Used standard SwiftUI templates and incremental build checks
Signing on another person's Mac	Device builds required a valid team, certificate, and keychain access	Selected the correct team and allowed codesign access with the Mac login password
Simulator runtime availability	Run destination was temporarily unavailable	Waited for runtime installation and verified compilation separately
App icon not visible	Default placeholder icon appeared	Created complete AppIcon assets and applied the DoseMate display name
tvOS project verification	Signing blocked a generic device build	Compiled with signing disabled to verify source

Classification: Public-SLIIT

		and resources
Git authentication	Terminal initially could not read credentials	Used the configured account and successfully pushed the repository

12.2 Current Limitations

- The iOS application uses UserDefaults rather than SwiftData or Core Data.
- The repository currently lacks dedicated unit-test and UI-test targets.
- The notification authorization result and scheduling errors are not presented to the user.
- Completion is recorded once per medicine per day, so multiple daily doses of the same medicine are not modelled separately.
- The tvOS application uses sample data and is not yet synchronized with the iOS application.
- There is no account system, cloud backup, caregiver role management, or encrypted cross-device sharing.
- The iOS Part A implementation does not yet integrate CoreML, ARKit, HealthKit, or Maps; local notifications are its principal advanced framework integration.
- The product is an educational prototype and does not provide medical diagnosis or treatment advice.

12.3 Future Development

1. Migrate medicine and completion data to SwiftData with schema migration and searchable history.
2. Add HealthKit integration for optional medication-related health context where supported and appropriate.
3. Add MapKit to locate nearby pharmacies and support refill planning.
4. Model multiple dose times and schedules such as selected weekdays or interval-based treatment.
5. Add XCTest unit tests, XCUITest flows, accessibility audits, and performance measurements.
6. Implement secure CloudKit synchronization between iOS and tvOS.
7. Add caregiver invitations, permission roles, and privacy-preserving notifications.
8. Add exportable adherence reports for the user to share voluntarily with a healthcare professional.
9. Conduct user acceptance testing with older adults and caregivers and revise typography and focus behaviour from feedback.

13 Requirement Traceability

13.1 Part A Mapping

Table 15 Part A rubric mapping

Rubric Area	Evidence	Assessment

Classification: Public-SLIIT

Purpose and audience	Real-world medicine adherence problem and identified user groups	Covered
Advanced UI	Animation, transitions, swipe action, adaptive layouts, custom cards	Covered
SwiftUI customization	Reusable MedicineFormView, MedicineCardView, StatTile, theming	Covered
Navigation	TabView, NavigationStack, NavigationLink, modal edit flow	Covered
Persistence	Codable JSON in UserDefaults	Functional but less advanced than SwiftData/Core Data
Emerging technology	UserNotifications framework	Partial against examples such as HealthKit, Maps, CoreML, or ARKit
Branding and accessibility	Custom icons, logo, semantic colors, Dynamic Type, accessibility labels	Covered with further audit recommended
Testing and optimization	Manual scenarios and successful builds	Partial because automated test targets are absent
Documentation	This report, project README and pitch notes	Covered
Deployment readiness	GitHub repository, buildable projects, demo data	Covered subject to signing configuration

13.2 Part B Mapping

Table 16 Part B rubric mapping

Rubric Area	Evidence	Assessment
Innovative concept	Shared Apple TV household medicine dashboard and competitor comparison	Covered
Prototype functionality	Family cards, schedule navigation, urgent doses, progress, pitch screens	Covered at prototype level
Platform capability	tvOS focus navigation, large-screen layout, remote-friendly cards	Covered
User experience	High contrast, distance-readable type, simple member selection	Covered
Technical implementation	Separate tvOS SwiftUI project with structured models and reusable views	Covered
Pitch presentation	Problem, solution, market, revenue model, go-to-market strategy	Covered in app and PITCH.md

Classification: Public-SLIIT

14 Conclusion

DoseMate demonstrates a complete medicine reminder workflow on iOS and an innovative household extension on tvOS. The iOS application provides branded navigation, medicine CRUD operations, completion and supply tracking, history, local persistence, and recurring notifications. The tvOS prototype adapts the concept to a shared screen using focusable cards, large typography, family profiles, urgent dose indicators, and an integrated pitch presentation.

The project meets the core functional and presentation objectives of the assignment and provides a clear foundation for further development. Its strongest qualities are coherent cross-platform branding, focused user flows, reusable SwiftUI components, and a practical extension from individual reminders to family coordination. The most important future improvements are advanced persistence, a clearly recognized emerging iOS technology such as HealthKit or MapKit, automated testing, and secure synchronization between platforms.

Classification: Public-SLIIT

References

Apple Developer Documentation. SwiftUI. https://developer.apple.com/documentation/swiftui

Apple Developer Documentation. UserNotifications. https://developer.apple.com/documentation/usernotifications

Apple Developer Documentation. NavigationStack. https://developer.apple.com/documentation/swiftui/navigationstack

Apple Developer Documentation. Human Interface Guidelines. https://developer.apple.com/design/human-interface-guidelines

Apple Developer Documentation. Designing for tvOS. https://developer.apple.com/design/human-interface-guidelines/designing-for-tvos

Apple Developer Documentation. SwiftData. https://developer.apple.com/documentation/swiftdata

Apple Developer Documentation. HealthKit. https://developer.apple.com/documentation/healthkit

Apple Developer Documentation. MapKit. https://developer.apple.com/documentation/mapkit

SE4041 Mobile Application Design and Development Assignment 2 brief, 2026.

Classification: Public-SLIIT

Appendix A Build Instructions

A.1 Part A iOS Application

1. Open MedicineReminder.xcodeproj in Xcode.
2. Select the MedicineReminder scheme.
3. Choose an iPhone simulator or a connected iPhone.
4. For a physical device, choose a valid development team in Signing and Capabilities.
5. Press Run and allow notification permission when requested.

A.2 Part B tvOS Application

1. Open DoseMateTV/DoseMateTV.xcodeproj in Xcode.
2. Select the DoseMateTV scheme.
3. Choose an Apple TV simulator or configured Apple TV device.
4. Press Run and use the remote controls or simulator keyboard to move focus.
5. Select a family card to open a schedule and select Pitch to open the presentation.

Appendix B Repository Structure

```
MedicineReminder/
  MedicineReminder.xcodeproj
  MedicineReminder/
    Medicine.swift
    MedicineStore.swift
    NotificationManager.swift
    RootView.swift
    SplashView.swift
    TodayView.swift
    AddMedicineView.swift
    MedicineCardView.swift
    MedicineDetailView.swift
    HistoryView.swift
    Assets.xcassets
  DoseMateTV/
    DoseMateTV.xcodeproj
    DoseMateTV/
      DoseMateTVApp.swift
      TVModels.swift
      TVHomeView.swift
      TVPitchView.swift
      Assets.xcassets
    README.md
    PITCH.md
```


Classification: Public-SLIIT

Appendix C Suggested Viva Demonstration Order

1. Introduce the problem, target audience, and two-platform DoseMate concept.
2. Run the iOS application and show splash, Today, Add, Details, Edit, and History.
3. Create a medicine and explain how the notification request uses its UUID.
4. Mark the dose as taken and show progress, supply, and history updates.
5. Open the tvOS project and show the household dashboard and member schedule.
6. Open the Pitch screen and explain value proposition, market, revenue model, and future synchronization.
7. Show the GitHub repository and identify the iOS and tvOS project folders.
8. Discuss limitations honestly and explain the planned SwiftData, HealthKit or MapKit, testing, and CloudKit improvements.

Appendix D Student Completion Checklist

- Replace Insert Student ID on the cover page with the correct registration number.
- Confirm the student name on the cover page.
- Add lecturer or tutor name if required by the submission format.
- Capture final iOS and tvOS screenshots if the lecturer expects visual evidence inside the report.
- Run both applications on the demonstration Mac before the viva.
- Verify that the GitHub repository is accessible using the submitted URL.
- Review the AI assistance disclosure and retain the relevant chat interactions if requested.

Classification: Public-SLIIT